INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Introducción al Desarrollo de Nuevas Plataformas				
TÍTULO DE LA PRÁCTICA:	Plano interactivo de un ambiente con Canvas en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	07	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN:	17/10/2026	**HORA DE PRESENTACIÓN:**	23:59:00		
INTEGRANTE(s): - Jorge Luis Mamani Huarsaya - Yordano Hernan Boza Portilla				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS DE LOS EJERCICIOS PROPUESTOS

Link del repositorio GitHub: https://github.com/jorghee/introduction_new_platforms/tree/main/labs/lab07

Desarrollo de la Solución

Partiendo del ejercicio base guiado por la docente, la solución se expandió arquitecturalmente para superar las limitaciones de mantener datos “cableados” (*hardcoded*) y estados locales efímeros. Se diseñó un sistema desacoplado, modular y escalable, estructurado sobre cinco fases clave de ingeniería.

1. Del Enfoque Tradicional de Vistas al Canvas Declarativo de Compose

En el sistema clásico de vistas de Android (`android.view.View`), la creación de componentes gráficos personalizados requería crear subclases de `View`, sobrecargar el método imperativo `onDraw(canvas: android.graphics.Canvas)` y gestionar de forma manual y compleja los eventos táctiles en `onTouchEvent(event: MotionEvent)` [1]. Este paradigma implicaba lidiar con objetos mutables como `Paint`, coordinar llamadas manuales a `invalidate()` y mapear manualmente los punteros táctiles, lo que dificultaba la sincronización entre el estado interno y la pantalla.

Con Jetpack Compose, dicho flujo se simplifica drásticamente mediante el Composable `Canvas`. Este componente no hereda de ninguna clase de vista ni retiene estado interno pesado; en su lugar, expone un ámbito de dibujo (`DrawScope`) puramente declarativo. Al mutar cualquier dato de estado observado, Compose redibuja automáticamente el lienzo sin intervención manual. La detección de toques se traslada de forma homogénea a la cadena de modificadores mediante `Modifier.pointerInput`, aprovechando la función de suspensión `detectTapGestures` [2].

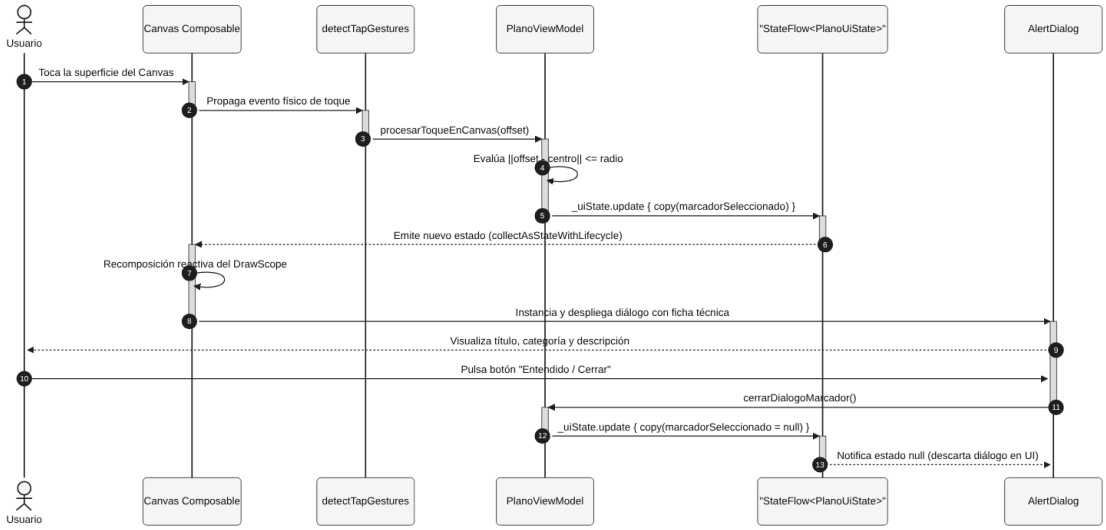


Figura 1: Diagrama de secuencia UML: Interacción temporal del toque físico, propagación al ViewModel y recomposición reactiva

2. Modelado de Ambientes del Campus y Renderizado Vectorial (Ejercicio Propuesto 1)

El primer ejercicio propuesto requirió ampliar el plano para representar la visualización interior de al menos 4 ambientes reales del campus universitario, integrando rectángulos, muros divisorios y etiquetas legibles visibles en el propio plano. Se seleccionó el primer piso del Pabellón EPIS (UNSA), modelando cuatro zonas críticas:

- **Lab 101 - Redes y Telecomunicaciones:** Dotado de racks y equipamiento de infraestructura de red.
- **Lab 102 - Desarrollo de Software:** Equipado con estaciones de alta gama para desarrollo móvil y computación en la nube.
- **Aula Magna / Sala de Grados:** Auditorio principal para defensas de tesis y conferencias magistrales.
- **Patio Central y Área de Conectividad:** Zona de libre esparcimiento académico con cobertura institucional de Eduroam.

Para cumplir con la exigencia de que cada ambiente cuente con su etiqueta visible sobre el plano, se utilizó la API moderna de tipografía en Canvas introducida en Jetpack Compose: `rememberTextMeasurer()` y `drawText()` [3]. En lugar de superponer torpemente Composables `Text` fuera del lienzo, `drawText` permite rasterizar texto vectorial directamente sobre las coordenadas relativas del `DrawScope`. Se calculó dinámicamente la posición del texto para garantizar su centrado horizontal dentro del tercio superior de cada rectángulo delimitador:

```
val textLayout = textMeasurer.measure(
    text = ambiente.etiqueta,
    style = TextStyle(fontSize = 10.sp, fontWeight = FontWeight.Bold, color = Color(0xFF2E1E22))
)
val labelX = ambiente.topLeft.x + (ambiente.size.width - textLayout.size.width) / 2f
drawText(textLayoutResult = textLayout, topLeft = Offset(labelX, ambiente.topLeft.y + 12f))
```

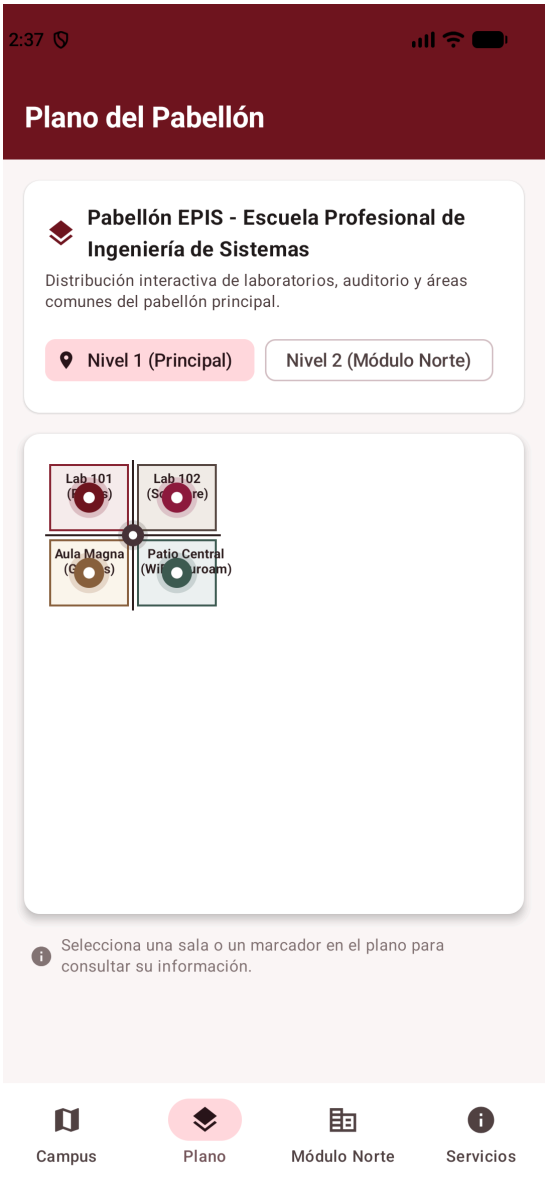


Figura 2: Visualización en el emulador del plano interactivo del Pabellón EPIS con 4 ambientes y etiquetas vectoriales visibles

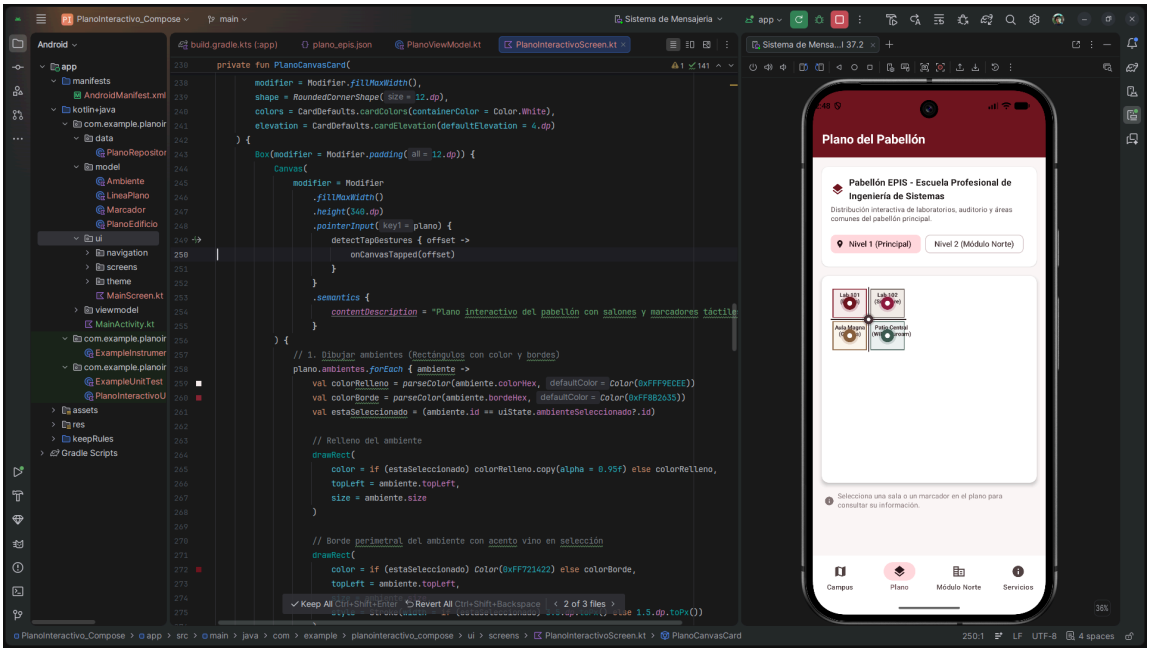


Figura 3: Entorno de desarrollo Android Studio: Implementación del `Canvas` y renderizado de primitivas en `PlanoInteractivoScreen.kt` junto al emulador activo

3. Persistencia Desacoplada y Fuente de Datos en Assets (Ejercicio Propuesto 2)

Mantener las coordenadas, dimensiones, colores y fichas informativas de los ambientes escritas de forma estática en el código fuente de Kotlin representa una mala práctica de desarrollo que dificulta el mantenimiento del plano. Por ello, en concordancia con el patrón de persistencia visto en el Laboratorio 03, se externalizó toda la información arquitectónica hacia el archivo local `app/src/main/assets/plano_epis.json`.

Para acceder a esta fuente de datos, se construyó la clase `PlanoRepository`. Este componente interactúa con el `AssetManager` de Android para abrir el flujo de entrada (`InputStream`), leer el documento estructurado y deserializar los arreglos de ambientes, líneas y marcadores utilizando `org.json.JSONObject`. Esta estrategia garantiza que cualquier cambio en la distribución del campus o en la ubicación de los salones pueda corregirse modificando únicamente el archivo JSON, sin necesidad de recompilar la lógica visual ni alterar los Composables.

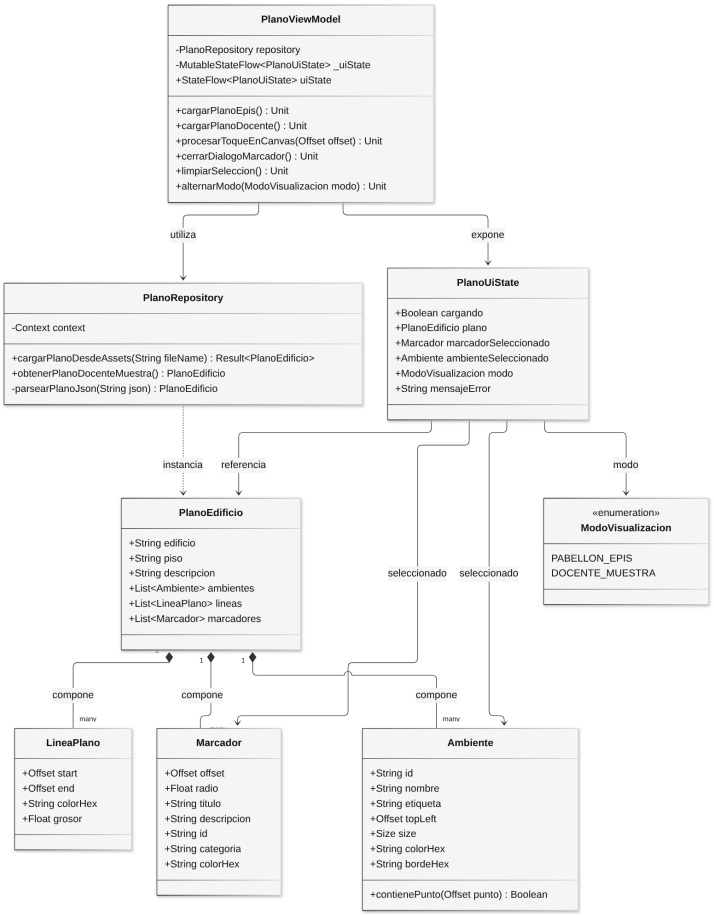


Figura 4: Diagrama de clases UML: Arquitectura estructural, modelos de dominio y desacoplamiento mediante el repositorio

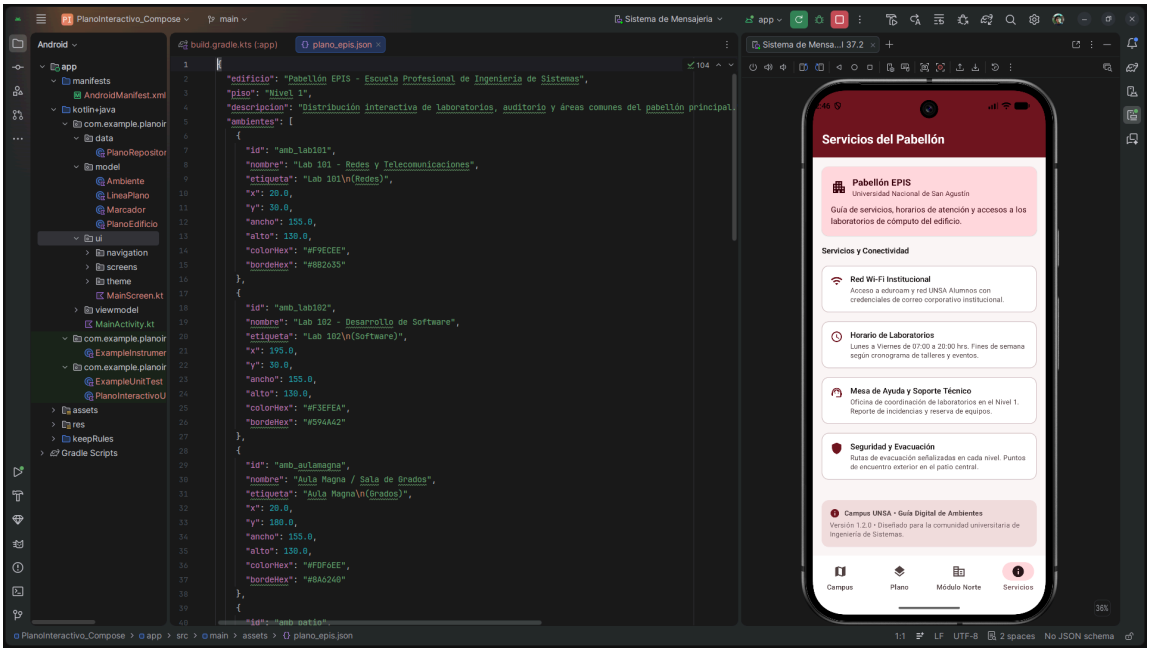


Figura 5: Estructura del archivo `plano_epis.json` en `app/src/main/assets` en Android Studio junto a la ejecución de la aplicación

4. Centralización Reactiva en PlanoViewModel y Algoritmo de Detección de Toques (Ejercicio Propuesto 3)

En el ejercicio resuelto por la docente, el estado (`marcadores` y `marcadorSeleccionado`) residía internamente en el Composable mediante `remember` y `mutableStateOf`. Siguiendo los principios de arquitectura limpia y Unidirectional Data Flow (UDF) profundizados en el Laboratorio 05, se trasladó la totalidad del estado hacia un `ViewModel` denominado `PlanoViewModel` [4].

La clase `PlanoViewModel` hereda de `AndroidViewModel`, lo que le permite acceder de manera segura al contexto de aplicación para el `PlanoRepository` sin ocasionar fugas de memoria (*memory leaks*). El estado se encapsula de forma inmutable a través de un `StateFlow<PlanoUiState>`. Al interactuar con el lienzo, el Composable captura las coordenadas físicas del toque y las despacha al ViewModel mediante la función `procesarToqueEnCanvas(offset)`.

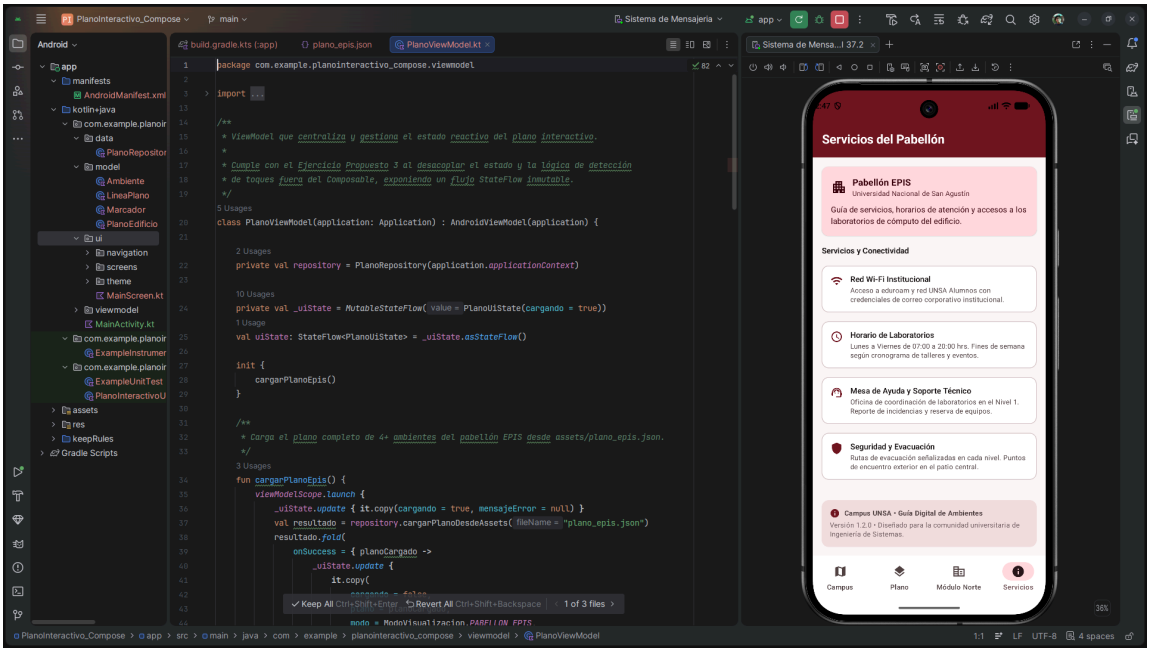


Figura 6: Gestión del estado reactivo inmutable y emisión con `StateFlow` en `PlanoViewModel.kt` dentro de Android Studio

El procesamiento del toque implementa un esquema de prioridad en dos niveles:

1. **Colisión Euclidiana (Marcadores Circulares):** Se itera la colección de marcadores evaluando si la distancia entre el toque y el centro del círculo es menor o igual a su radio: $\sqrt{(x_{\text{toque}} - x_{\text{centro}})^2 + (y_{\text{toque}} - y_{\text{centro}})^2} \leq r$ Si existe colisión, se actualiza `marcadorSeleccionado` en el estado, lo que gatilla la apertura inmediata del `AlertDialog` con la información técnica del ambiente.
2. **Colisión AABB (Recintos Rectangulares):** Si el toque no coincide con ningún marcador, se evalúa si las coordenadas caen dentro de los límites de algún ambiente (`x in [x1, x2]` e `y in [y1, y2]`). En caso afirmativo, se asigna `ambienteSeleccionado`, provocando un repintado con trazo de borde resaltado (`Stroke` de 3.5 dp) en el Canvas.

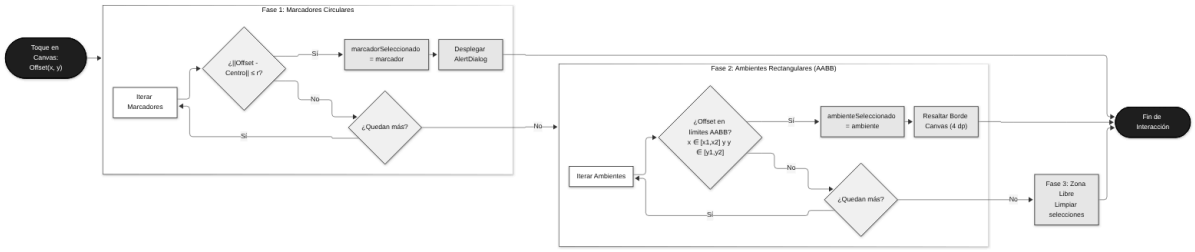


Figura 7: Diagrama de flujo del algoritmo de resolución geométrica de colisiones (Distancia Euclidiana y límites AABB)

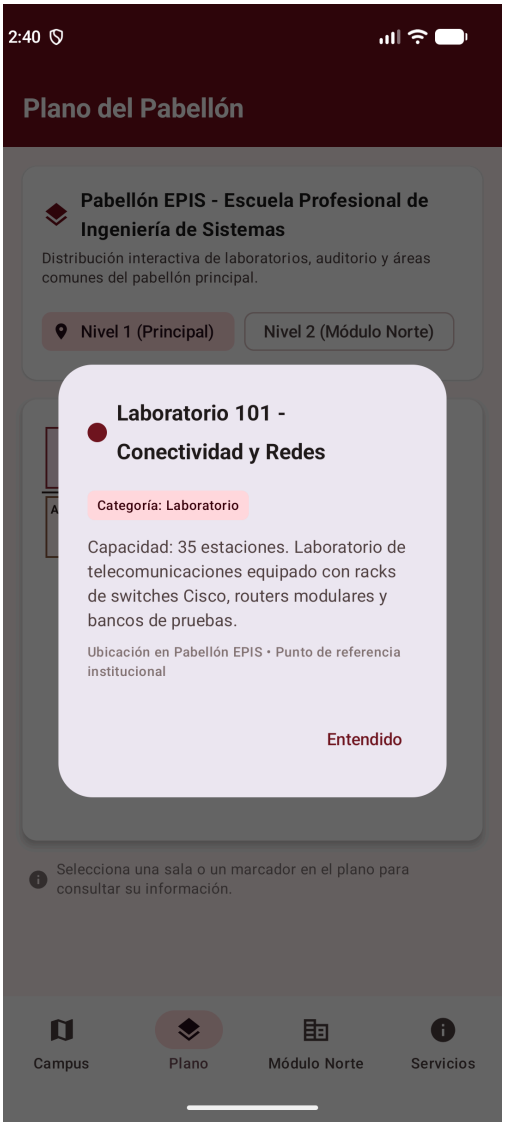


Figura 8: Diálogo emergente `AlertDialog` mostrando la ficha técnica al interactuar con un marcador circular

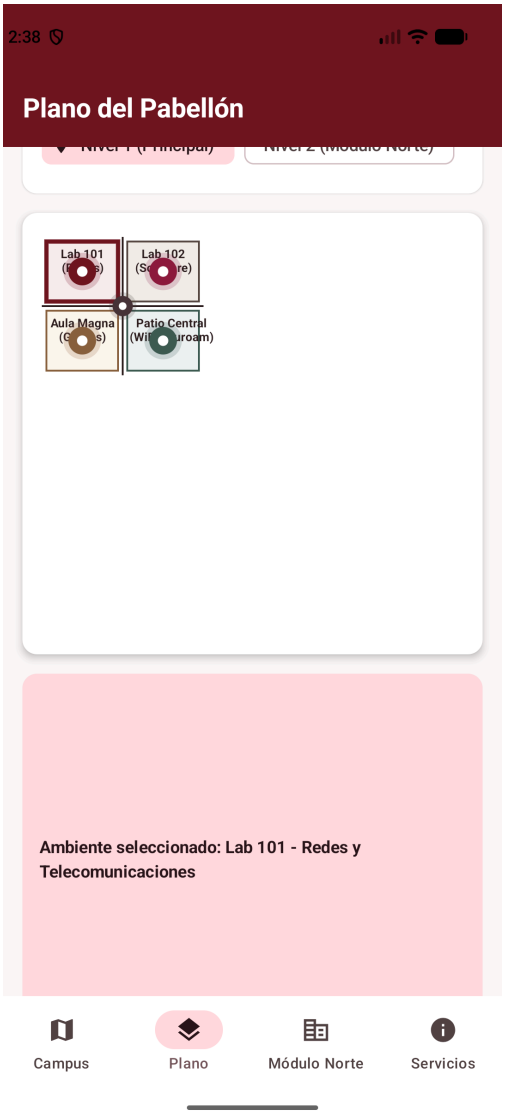


Figura 9: Resaltado perimetral y panel informativo contextual al pulsar sobre la superficie de un ambiente rectangular

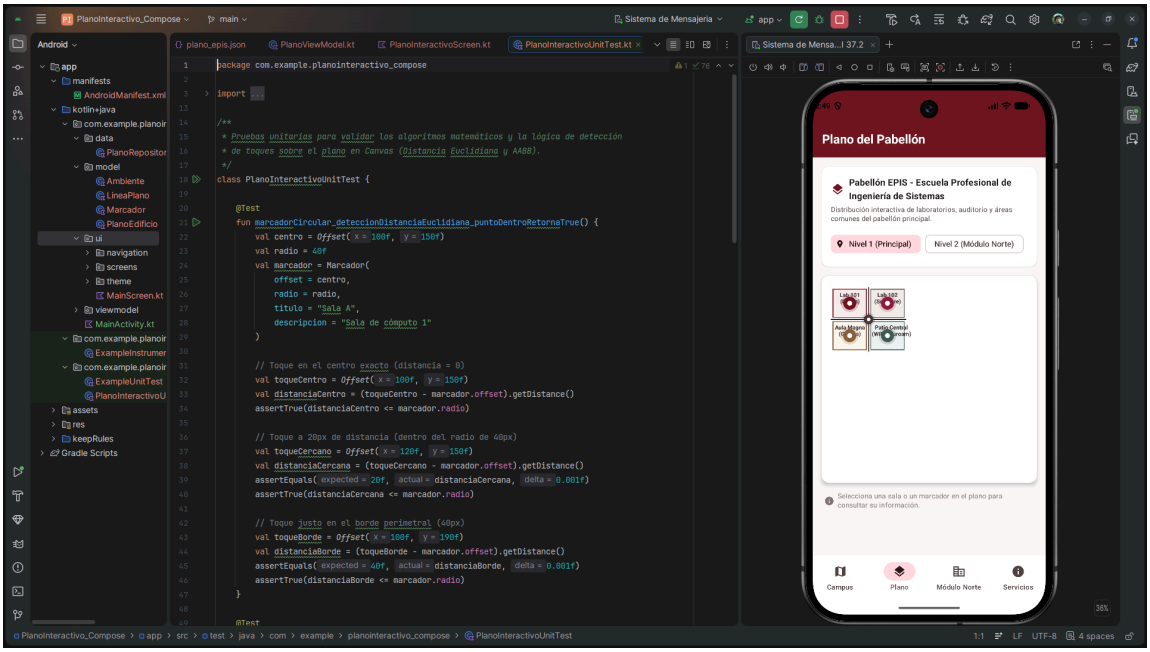


Figura 10: Pruebas unitarias automatizadas en `PlanoInteractivoUnitTest.kt` en Android Studio validando la detección euclidiana y AABB

5. Integración Multi-pantalla y Accesibilidad desde el Mapa (Ejercicio Propuesto 4)

El último ejercicio propuesto requirió vincular el plano interactivo como una nueva pantalla accesible desde el flujo de navegación de la aplicación (continuando la pantalla `MapaScreen` iniciada en el Laboratorio 05). Para ello, se configuró un `NavHost` central orquestado por un `Scaffold` con barra de navegación inferior (`NavigationBar`) [5].

Se diseñó la pantalla `MapaCampusScreen`, la cual ofrece una panorámica del campus de Ingenierías de la UNSA e incluye una tarjeta destacada interactiva para el Pabellón EPIS. Al pulsar el botón “Explorar Plano del Pabellón”, el `NavController` ejecuta la transición fluida hacia `PlanoInteractivoScreen`, reutilizando la misma instancia del `PlanoViewModel` compartido. Adicionalmente, la barra inferior cuenta con pestañas dedicadas para alternar entre el mapa general, el plano EPIS, el módulo norte y los servicios del edificio.



Figura 11: Pantalla `MapaCampusScreen` con la tarjeta interactiva que da acceso directo al plano del pabellón



Figura 12: Pantalla `InfoScreen` con el directorio y catálogo de servicios institucionales del pabellón

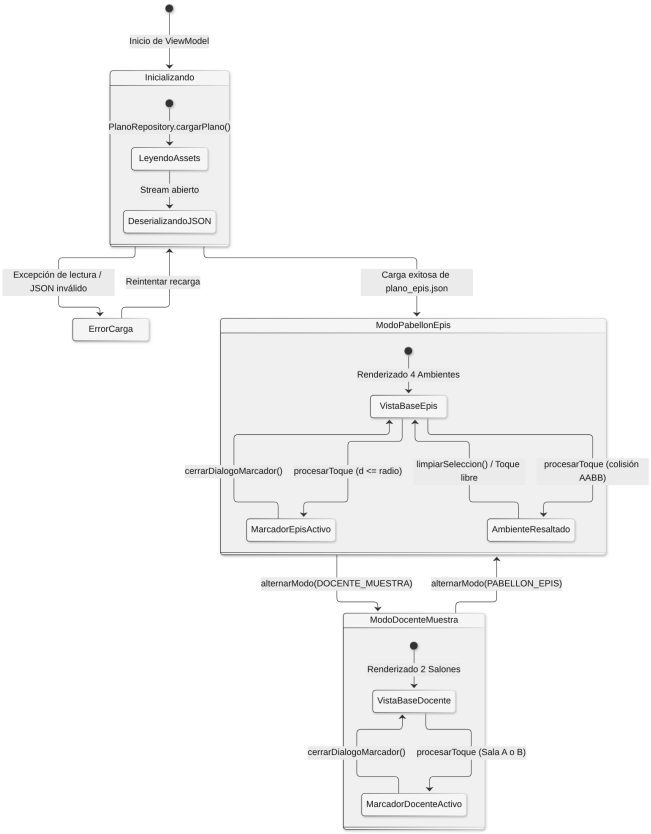


Figura 13: Diagrama de estados UML: Ciclo de vida y transiciones de la interfaz gestionadas por `PlanoViewModel`

CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (plano de un ambiente dibujado con Canvas, donde tocar un marcador abre un diálogo con su información)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, se resolvió de forma íntegra y rigurosa el ejercicio propuesto por la docente. Para evidenciar su exactitud y permitir una evaluación comparativa inmediata frente a los ejercicios propuestos ampliados, se implementó la pantalla dedicada `DocentePlanoScreen.kt` (accesible desde la tercera pestaña de la barra de navegación inferior).

Esta implementación replica de manera exacta los Pasos 1 al 11 de la guía:

- Creación de la estructura de datos `Marcador` con atributos `offset`, `radio`, `titulo` y `descripcion`.
- Dibujo de los dos salones rectangulares (`Salón 1` en azul claro `#BBDEFB` y `Salón 2` en verde menta `#C8E6C9`) mediante `drawRect`.
- Trazado del tabique o muro divisorio central en gris oscuro mediante `drawLine`.
- Renderizado de dos marcadores circulares de radio $r = 40$ px en rojo mediante `drawCircle`.
- Detección de toques con `Modifier.pointerInput(Unit) { detectTapGestures { ... } }`, calculando la distancia euclidiana `(offset - marcador.offset).getDistance() <= marcador.radio`.

- Apertura del componente modal `AlertDialog` con el título y la descripción del salón pulsado al cumplirse la condición de contacto.

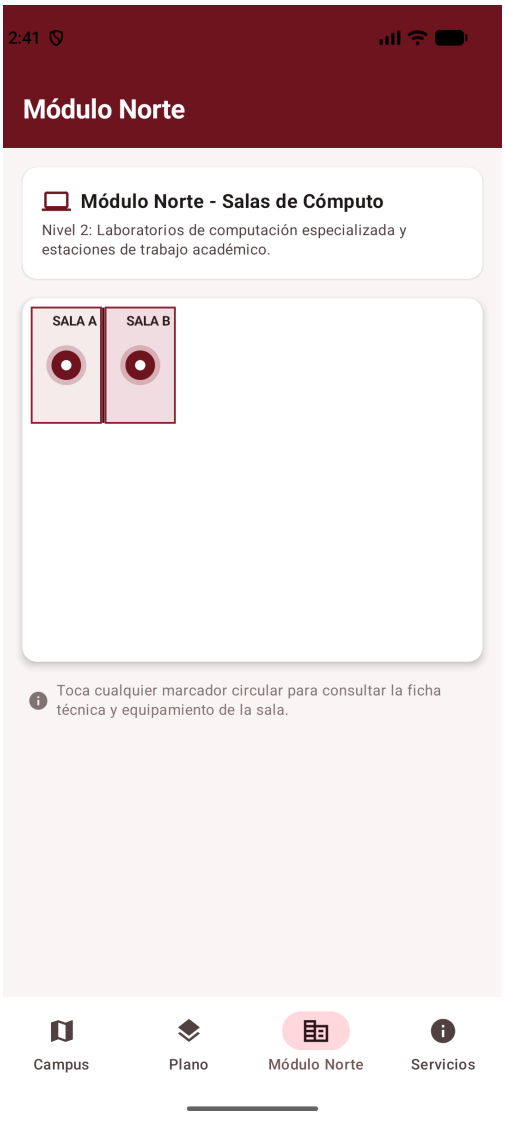


Figura 14: Plano base del ejercicio docente con 2 salones, muro divisorio y marcadores interactivos

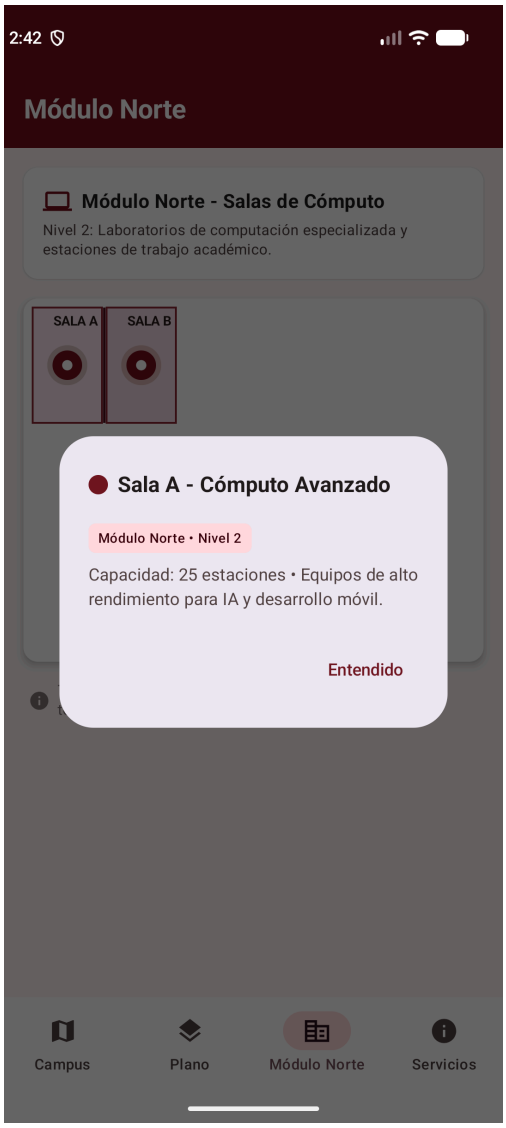


Figura 15: Diálogo modal informativo `AlertDialog` abierto tras tocar el marcador de la Sala A

2. En tu propia implementación, ¿cómo calculaste si un toque cayó dentro de un marcador circular? ¿Qué otra estrategia se te ocurre para detectar toques sobre formas no circulares, como un rectángulo o un polígono?

Cálculo implementado para marcadores circulares: En nuestra solución, el evento de toque entregado por el callback `detectTapGestures` suministra un vector de posición $P_{\text{toque}} = (x_t, y_t)$ en píxeles locales del Canvas. Para verificar si este punto se encuentra en el interior del marcador de centro $C = (x_c, y_c)$ y radio r, se empleó el principio de la **Distancia Euclidiana**:

$$d = \|P_{\text{toque}} - C\| = \sqrt{(x_t - x_c)^2 + (y_t - y_c)^2}$$

En Jetpack Compose, la clase `Offset` implementa la sobrecarga del operador de resta (`-`), retornando un vector diferencial, sobre el cual se invoca la función de extensión `.getDistance()`. Si la condición $d \leq r$ se satisface, el toque colisiona con el marcador.

Optimización computacional: En entornos de alto rendimiento o con cientos de marcadores, el cálculo de la raíz cuadrada puede omitirse comparando directamente las distancias al cuadrado para reducir ciclos de reloj en la CPU:

$$(x_t - x_c)^2 + (y_t - y_c)^2 \leq r^2$$

Estrategias de detección para formas no circulares:

1. **Formas Rectangulares (Axis-Aligned Bounding Box - AABB):** Para salones o recintos rectangulares paralelos a los ejes de coordenadas (definidos por su punto superior izquierdo $T = (x_0, y_0)$, ancho W y alto H), se verifica si el punto cae dentro del intervalo cerrado de ambos ejes cartesianos:

   $$x_0 \leq x_t \leq (x_0 + W) \quad \wedge \quad y_0 \leq y_t \leq (y_0 + H)$$

   Esta estrategia fue implementada en nuestra clase `Ambiente` mediante la función `contienePunto(punto: Offset)`.
2. **Polígonos Arbitrarios (Algoritmo de Ray Casting / Regla Par-Impar):** Si el plano contiene salas trapezoidales, polígonos irregulares o patios diagonales, se traza un rayo virtual horizontal infinito que parte desde el punto de toque (x_t, y_t) hacia la derecha ($x \to \infty$). Se cuentan cuántas veces dicho rayo interseca los segmentos de arista del polígono:
   - Si el número de intersecciones es **impar**, el punto se encuentra **dentro** del polígono.
   - Si el número de intersecciones es **par** o cero, el punto se encuentra **fuera**.
3. **Uso de Rutas Vectoriales (`Path`) y Regiones Gráficas:** Otra alternativa en Android consiste en construir un objeto `androidx.compose.ui.graphics.Path` que encierre el contorno del polígono y evaluar la contención mediante `android.graphics.Region`, asociando una máscara rasterizada o empleando la función `Path.asAndroidPath()` junto a `Region.setPath(...)` para consultar si `region.contains(x, y)` es verdadero.

3. En forma individual, comenta en el video de referencia qué otra forma se te ocurre para lograr un dibujo interactivo similar sin depender directamente de Canvas y DrawScope. (Captura del comentario; uno por integrante del grupo).

Propuesta del estudiante Jorge Luis Mamani Huarsaya: ““Una alternativa muy potente en Jetpack Compose para construir este tipo de planos sin depender de `Canvas` y `DrawScope` consiste en emplear una arquitectura de **Composición Jerárquica basada en `Box` con modificadores de posicionamiento absoluto o porcentual** (`Modifier.offset` o `Modifier.layout`). En lugar de pintar primitivas gráficas en un único lienzo indivisible, cada ambiente y marcador puede representarse como un Composable nativo independiente (por ejemplo, `Surface`, `Card` o `IconButton`).

Esta técnica ofrece dos ventajas críticas: en primer lugar, se aprovecha de forma directa el sistema nativo de eventos y clics de Compose (`Modifier.clickable`), eliminando por completo la necesidad de programar manualmente funciones matemáticas de colisión geométrica euclidiana. En segundo lugar, los componentes conservan propiedades inherentes de accesibilidad (soporte nativo para lectores de pantalla *TalkBack* y semántica de UI), algo que un `Canvas` estándar pierde al ser tratado como una caja negra de píxeles.“”

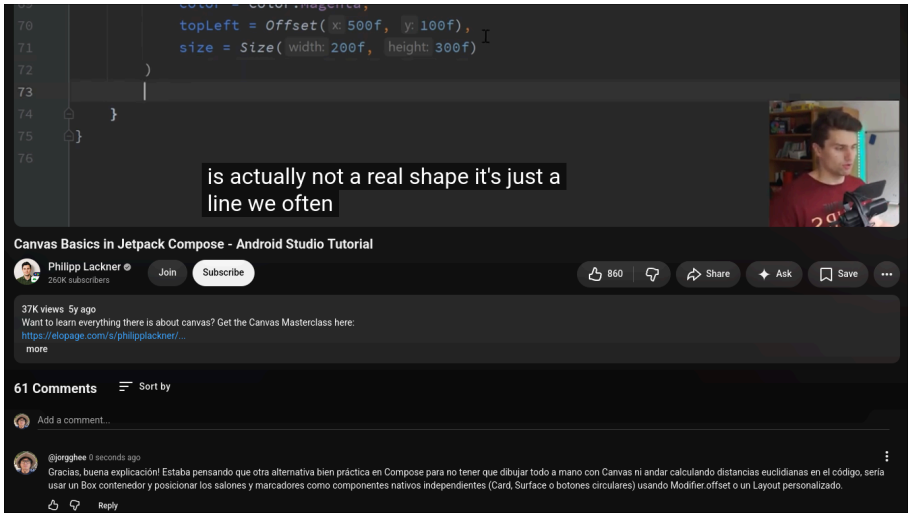


Figura 16: Evidencia del comentario publicado por Jorge Luis Mamani Huarsaya en el video tutorial de referencia

Propuesta del estudiante Yordano Hernan Boza Portilla: ““Otra alternativa sumamente escalable para lograr un plano interactivo sin depender directamente del Composable `Canvas` es la integración de **Gráficos Vectoriales Escalables (SVG) interactivos a través de `ImageVector` o bibliotecas de renderizado dinámico como AndroidSVG o Coil-SVG**.

Bajo este enfoque, el plano del campus se modela previamente en herramientas de diseño vectorial (como Figma o Inkscape), asignando identificadores semánticos (`id="sala_101"`, `id="marcador_redes"`) a las etiquetas `<path>` y `<circle>` del archivo SVG. Al cargar el documento en la aplicación, la biblioteca parsea la jerarquía del DOM vectorial; mediante un interceptor de eventos táctiles, se captura el nodo XML tocado evaluando directamente la ruta vectorial del objeto. Esto permite un escalado vectorial infinito sin pérdida de nitidez, soporte nativo para zoom/pellizco (`pinch-to-zoom`) y estilos gráficos complejos (degradados, texturas y sombras) sin tener que escribir cientos de líneas de código de trazado manual en `DrawScope`.””

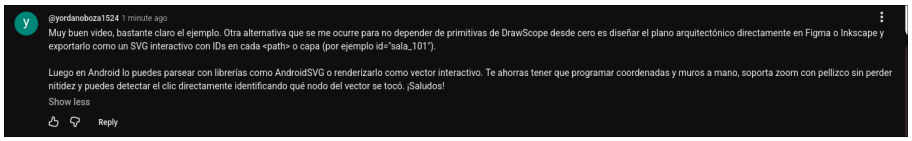


Figura 17: Evidencia del comentario publicado por Yordano Hernan Boza Portilla en el video tutorial de referencia

CONCLUSIONES

- **Eficiencia y simplicidad del paradigma gráfico declarativo:** La adopción del Composable `Canvas` y la API de `DrawScope` en Jetpack Compose demostró ser notablemente superior a la arquitectura clásica de vistas (`View.onDraw`). Se eliminó la sobrecarga de mantener instancias pesadas de `Paint` y sincronizaciones manuales con `invalidate()`, logrando un pipeline gráfico limpio, donde cualquier mutación en el estado reactivo gatilla un redibujado instantáneo de la superficie de dibujo.
- **Dominio del sistema de coordenadas y detección matemática de colisiones:** Al tratarse de una única superficie de dibujo continua sin elementos hijos en el árbol semántico, la interacción táctil exige el cálculo matemático explícito

de coordenadas locales. La implementación combinada del cálculo de distancia euclidiana ($d \leq r$) para pines circulares y la prueba de límites en cajas alineadas con los ejes (AABB) para salas rectangulares permitió estructurar un sistema de interacción táctil con prioridades bien delimitadas.

- **Madurez arquitectónica mediante ViewModel y deserialización local:** Externalizar el plano arquitectónico hacia un archivo estructurado `plano_epis.json` dentro de `assets/` y centralizar la gestión de eventos en `PlanoViewModel` representó una mejora determinante sobre el enfoque guiado inicial. Este desacoplamiento cumple con el patrón Unidirectional Data Flow (UDF), facilitando el mantenimiento a largo plazo de la aplicación, ya que añadir nuevos salones o modificar coordenadas no requiere alterar el código fuente de Kotlin.
- **Integración armónica en flujos multi-pantalla:** La vinculación del plano arquitectónico como una pantalla secundaria accesible desde una tarjeta destacada en `MapaCampusScreen` completó el ciclo de navegación iniciado en el Laboratorio 05. El uso de rutas selladas en `Screen` y la integración fluida en un contenedor `Scaffold` con `NavigationBar` consolidan una arquitectura móvil modular, robusta y escalable.

REFERENCIAS

Bibliography

[1] Google Developers, “Graphics in Compose | Android Developers.” Accessed: Oct. 17, 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/graphics/draw/overview

[2] Google Developers, “Pointer input in Compose | Android Developers.” Accessed: Oct. 17, 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/touch-input/pointer-input

[3] Google Developers, “Text in Compose - Measuring and drawing text in Canvas.” Accessed: Oct. 17, 2026. [Online]. Available: https://developer.android.com/reference/kotlin/androidx/compose/ui/text/TextMeasurer

[4] Google Developers, “ViewModel Overview | Android Developers.” Accessed: Oct. 17, 2026. [Online]. Available: https://developer.android.com/topic/libraries/architecture/viewmodel

[5] Google Developers, “Navigating with Compose | Android Developers.” Accessed: Oct. 17, 2026. [Online]. Available: https://developer.android.com/jetpack/compose/navigation